RaceDay

RaceDay is a race-event platform. Organisers create events and categories and record results; Participants browse events, enrol and view their own results.

Part 2: an ASP.NET Core Web API (C#) with SQL Server, Entity Framework Core, session-based role authentication, Swagger, unit tests and GitHub Actions CI/CD. The code is in the `Part2/` folder.

Roles

Role	What they can do
Organiser	Create, update and delete their own events. Add categories to their events. See everyone enrolled in their events. Record finish times and positions after an event. View and update their profile.
Participant	Browse events and categories. Enrol in an event by choosing a category (and cancel the enrolment). View their enrolments and their own results. View and update their profile.

Organiser functions return **403** for Participants and Participant functions return **403** for Organisers. Protected endpoints return **401** when nobody is logged in.

Registering as an Organiser: anyone can register as a Participant. Registering as an Organiser needs the secret `organiserCode` (set in `appsettings.json` under `Registration:OrganiserCode`, default `RACEDAY-ORG-2026`), so users cannot simply promote themselves.

Requirements

- .NET 8 SDK
- SQL Server (LocalDB comes with Visual Studio) or any SQL Server instance
- Visual Studio 2022 (optional)

Setup and run

1. Clone the repository.

2. Open `Part2/RaceDay.sln` in Visual Studio (or use the terminal).
3. Check the connection string in `Part2/RaceDay.Api/appsettings.json` (default: LocalDB, database `RaceDayDB`).
4. Run the API:

```
cd Part2/RaceDay.Api
dotnet run
```

5. Open **http://localhost:5080/swagger**. The database and the `Roles` rows (1 = Organiser, 2 = Participant) are created on start-up.

Database (EF Core code-first)

The schema is built from the EF Core model, using the table and column names from the Part 1 SQL script. To use migrations (recommended for submission):

```
cd Part2/RaceDay.Api
dotnet tool install --global dotnet-ef
dotnet ef migrations add InitialCreate
dotnet ef database update
```

If the database was already created automatically, drop it first (`dotnet ef database drop`). `Part2/database/schema-part2.sql` is the Part 1 script plus the two new Events columns.

Try it in Swagger

1. `POST /api/auth/register` (Participant: `roleId` 2, or Organiser: `roleId` 1 with `organiserCode`).
2. `POST /api/auth/login`. The session cookie is stored by the browser, so the other endpoints now work in Swagger UI.
3. `POST /api/auth/logout` ends the session.

Authentication design

- Session-based: on login the server stores the user id, name and role in a server-side session (30 minutes idle timeout) and sends an HttpOnly session cookie.
- `SessionAuthenticationHandler` turns the session into the logged-in user, so `[Authorize]` and `[Authorize(Roles = "Organiser")]` work on the controllers.
- Passwords are hashed with salted PBKDF2 (SHA-256, 210,000 iterations). The

original password is never stored.

Endpoints

Routes from the Part 1 plan:

Method	Route	Access
POST	`/api/auth/register`	Public
POST	`/api/auth/login`	Public
GET	`/api/users/profile`	Any logged in
PUT	`/api/users/profile`	Any logged in
GET	`/api/events`	Public
GET	`/api/events/{id}`	Public
POST	`/api/events`	Organiser
PUT	`/api/events/{id}`	Organiser (own events)
DELETE	`/api/events/{id}`	Organiser (own events)
GET	`/api/events/{eventId}/categories`	Public
POST	`/api/events/{eventId}/categories`	Organiser (own events)
POST	`/api/enrolments`	Participant
GET	`/api/enrolments/my`	Participant
POST	`/api/results`	Organiser (own events)
GET	`/api/events/{eventId}/results`	Public

Added because the Part 2 brief needs them and the Part 1 plan did not list them:

Method	Route	Access
POST	`/api/auth/logout`	Any logged in
GET	`/api/events/{eventId}/enrolments`	Organiser (own events)

DELETE	`/api/enrolments/{id}`	Participant (cancels own enrolment)
GET	`/api/results/my`	Participant

Differences from Part 1 (please read before submitting)

- **Events:** the brief requires a distance and an event type (Run, Walk, Cycle) on each event. These were not in the Part 1 SQL script, so `Events.DistanceKM` and `Events.EventType` were added. Update your ERD to match.
- **ERD vs SQL script:** the Part 1 ERD (FirstName/LastName, Role column, EVENT_CATEGORY table) does not match the SQL script (FullName, Roles table, Categories under Events). The code follows the SQL script and the API plan.
- **Login:** the plan said login returns a token. The brief requires session management, so login returns the user and sets the session cookie instead.
- **Business rules added:** a participant can enrol in an event only once and only before it takes place; results can only be recorded after the event date; a finishing position is unique within an event.

Unit tests

xUnit tests run the real API in memory (EF Core InMemory database) and cover success and failure scenarios for registration and login, event management, role rejection, enrolments, results and password hashing.

```
cd Part2
dotnet test
```

CI/CD (GitHub Actions)

`.github/workflows/ci.yml` restores, builds and runs all tests on every push and pull request.

Green build screenshot: add your screenshot here after the first successful run.

```
![CI green build](docs/ci-green-build.png)
```

Demo video

YouTube (unlisted): **add your link here**

Git

Commit regularly with meaningful messages (at least 20 for Part 2) from your own GitHub account.